Real-Time Scheduler — Project Design Document

Rebecca Robinson Manu Geronimo Amin Charepoo

Basic Functionalities

The table below lists the main functionalities the project will implement, including a brief description, expected inputs and outputs, and each functionality's contribution to the overall system.

Functionality	**Description**	**Input**	**Output**	**Contribution**
Task Scheduling	Manages a priority queue of tasks, ensuring higher-priority tasks are completed before lower-priority ones.	Task object with a priority level	Tasks dequeued and executed in the correct order	Determines execution order across the entire system.
Button Interrupt Handling	Detects a button press and enqueues the corresponding task.	GPIO pin signal	Task enqueued	Allows user input to trigger tasks.
Timer-Based Interrupt	Uses a repeating hardware timer at a configurable interval and enqueues the given task each time it fires.	Timer interval (ms)	Task enqueued periodically	Enables periodic automated behaviour without user input.

Functionality	Description	Input	Output	Contribution
VIP Task Interrupt	When a VIP-priority task is enqueued, it stops the currently running task early, executes the VIP task, then resumes or drops the interrupted task as required.	VIP task object	Interrupted task dropped or re-queued at the front; VIP task executed	Ensures critical tasks are completed promptly.
Raspberry Pi Pico	Microcontroller board to which buttons, LEDs, and the LCD screen are connected.	Physical buttons, LEDs, LCD screen	Buttons light up and react on press; LCD shows system state in a human-readable way	Provides a tangible demonstration of the scheduler so the audience can visualise and understand it.
Button (connected to Pico)	A physical button that, when pressed, adds an element to the queue.	0 (unpressed) / 1 (pressed)	When the button is pressed, a task will be enqueued into our Real-Time Scheduler	Offers an easy physical and visual representation of the scheduler for the audience.
LEDs (connected to Pico)	Two LEDs that react to button presses while the code runs.	Button press	One LED will light up when the button is pressed to show the queue, while the other LED will turn on to show that the entire system is on and working.	Provides additional visual feedback for the audience.

OOP Design Table

The table below describes the Object-Oriented Programming structure of the system, identifying classes, attributes, methods, access modifiers, and data types.

Class: Task

Attribute / Method	Access / Data Type	Description
`name`	`private / string`	Name of the task.
`priority`	`private / priority_level`	Execution order (enum).
`behavior`	`private / interrupt_behavior`	What happens if interrupted by a VIP task (enum).
`queue`	`protected / Queue&`	Reference to the queue so the task can enqueue itself.
`getPriority()`	`public / priority_level`	Returns the task's priority.
`getBehavior()`	`public / interrupt_behavior`	Returns the task's interrupt behaviour.
`getName()`	`public / string`	Returns the task's name.
`execute()`	`public virtual / void`	Executes the task's action.

Class: Queue

Attribute / Method	Access / Data Type	Description
`tasks`	`private / Task*[]`	Array holding pointers to queued tasks.
`head`	`private / volatile int`	Index of the front of the queue.
`tail`	`private / volatile int`	Index of the back of the queue.
`full()`	`bool`	Returns `true` if the queue is at capacity.
`empty()`	`bool`	Returns `true` if the queue has no tasks.
`enqueue()`	`void`	Adds a task to the back of the queue and checks if it is a VIP priority.
`dequeue()`	`Task*`	Scans the queue for the highest-priority task and returns it.

Attribute / Method	Access / Data Type	Description
`push_front()`	`void`	Inserts a task at the front of the queue.
`push_back()`	`void`	Inserts a task at the back of the queue.

Class: ButtonTask

Attribute / Method	Access / Data Type	Description
`button_pin`	`private / int`	GPIO pin number.
`action`	`private / void(*)()`	Function pointer executed when the button is pressed.
`instance`	`private static / ButtonTask*`	Static pointer to itself so the ISR can access the instance.
`setup()`	`public / void`	Initialises the GPIO pin and registers the button callback.
`execute()`	`public / void`	Calls the assigned function.
`button_isr()`	`private static / void`	Debounces and enqueues the task on a button press.

Class: TimerTask

Attribute / Method	Access / Data Type	Description
`interval_ms`	`private / uint32_t`	Interval for the timer interrupt in milliseconds.
`timer`	`private / repeating_timer`	Pico SDK struct holding internal timer state.
`action`	`private / void(*)()`	Function pointer executed when the timer fires.
`start()`	`private / void`	Starts the repeating timer.
`stop()`	`public / void`	Cancels the repeating timer.
`execute()`	`public / void`	Calls the assigned function.
`timer_callback()`	`public static / bool`	Enqueues the task each time the timer fires.

Class: DisplayTask

Attribute / Method	Access / Data Type	Description
`Message`	`private / string`	Stores the string needed to be displayed.
`char_index`	`private / int`	Tracks where in the message the display is.
`interval_ms`	`private / unit32_t`	How long to wait between displaying each character.
`timer`	`private / repeating_timer`	Pico SDK struct that holds internal timer state
`timer_flag`	`volatile private / bool`	Set by ISR each interval to tell the run loop it can display a character
`setMessage()`	`public / void`	Takes in a string and sets the message variable
`display()`	`public / void`	Displays the next character
`clear()`	`public / void`	Clears the screen
`run()`	`public / void`	Keeps calling display() every time timer_flag is true and checks if there is a VIP task pending.

System Overview

Input

The following data is provided to the system:

- Button press (GPIO signal from the physical button on the Pico).
- Repeating hardware timer firing at a fixed interval.

Process

The system processes input through the following steps:

1. A hardware interrupt fires (button press or timer tick).
2. The Interrupt Service Routine (ISR) runs and enqueues the task.
3. `enqueue()` places the task at the back of the queue and checks whether it carries VIP priority.
4. A currently running task periodically checks for a pending VIP task; if one exists, the running task stops early.
5. The main loop dequeues and executes the highest-pr iority task.
6. After a VIP task finishes, the scheduler checks whether the interrupted task should be requeued at the front (resume) or dropped, depending on its configured `interrupt_behavior`.

Output

The system produces the following outputs:

- GPIO pin toggled (LED on/off) in response to task execution.
- Serial debug output via USB confirming task execution, interruptions, and task resumptions.
- Correct priority ordering verified through observed execution sequence.

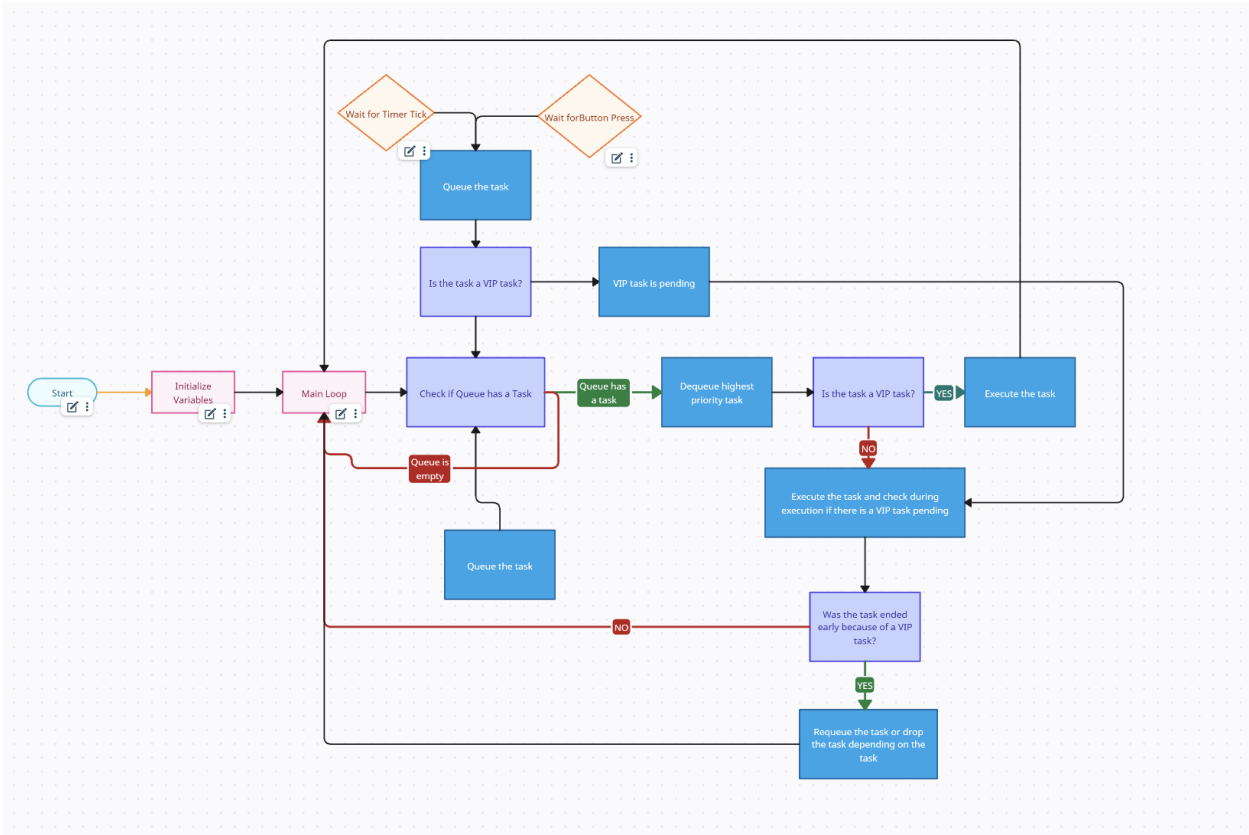

Figure 1: System Overview Diagram

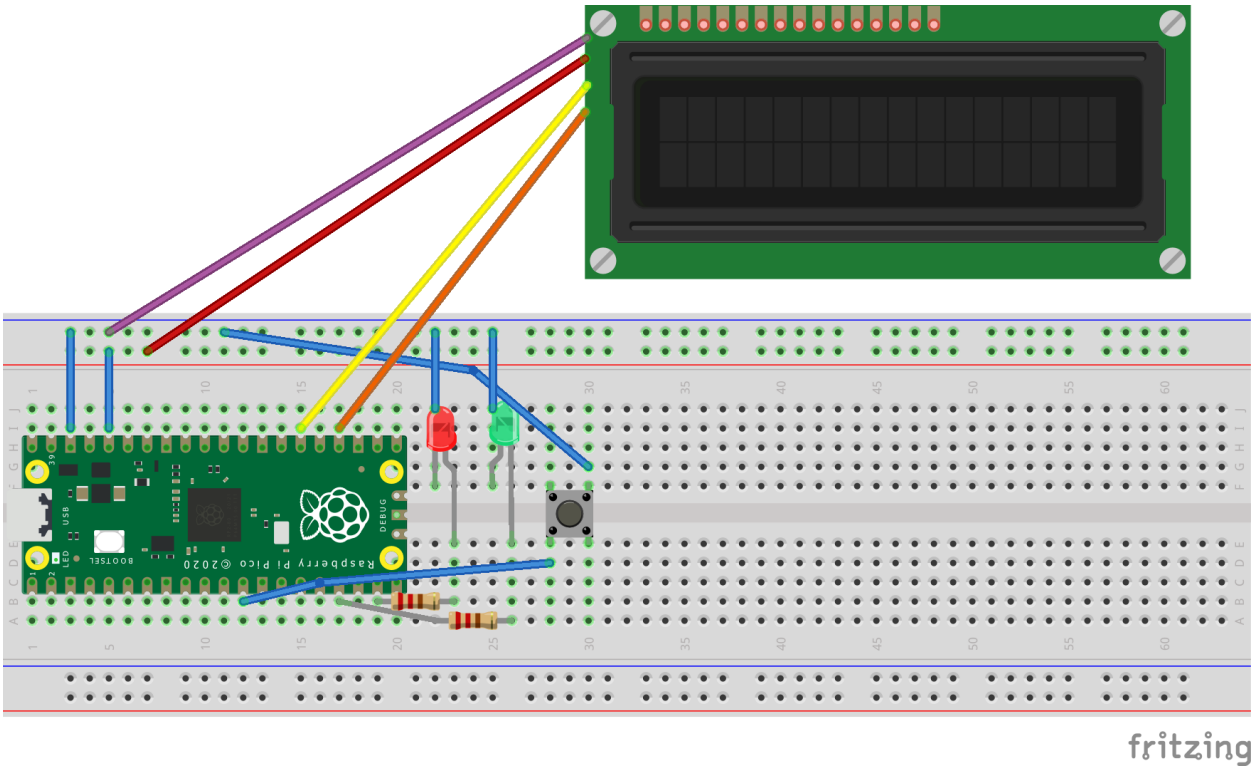

Figure 2: System Circuit Diagram